Data Catalog, Policy Tags & Column-Level Security

A Practical Handbook for BigQuery Data Governance on Google Cloud

Covers: Data Catalog concepts • Taxonomies & Policy Tags • Column-level access control • Dynamic data masking • IAM roles • A full hands-on demo you can run yourself in a GCP project (console, gcloud/bq CLI, and a companion Colab notebook)

Prepared for GCP Data Engineering Training

Reflects Google Cloud naming as of July 2026 — Knowledge Catalog / Dataplex Universal Catalog

Table of Contents

1. What Is a Data Catalog?

> **IMPORTANT:** As of July 2026: the standalone **Data Catalog** product was deprecated on January 30, 2026, with a phased API shutdown starting June 1, 2026. Its features live on inside **Dataplex Universal Catalog**, which Google renamed **Knowledge Catalog** on April 10, 2026. The APIs, IAM role names, and `gcloud data-catalog` CLI commands are unchanged — only the console name and branding moved. This handbook uses 'Data Catalog' and 'Knowledge Catalog' interchangeably, and every command shown works today.

A **data catalog** is a searchable inventory of your organization's data assets — tables, files, streams, dashboards, ML models — along with metadata that describes what each asset is, who owns it, how sensitive it is, and how it connects to other assets. Think of it as a “Google Search for your company's data” combined with a governance layer that controls who can see what.

On Google Cloud, this capability is delivered by **Knowledge Catalog** (built on Dataplex), which automatically scans and indexes metadata from services like BigQuery, Cloud Storage, Pub/Sub, Spanner, Cloud SQL, Dataproc Metastore, and Vertex AI/Gemini Enterprise. It gives you three things a beginner should understand clearly:

- **Discovery** — search across all your data assets by name, description, tag, or business term, so people don't have to ask a colleague “which table has customer emails?”
- **Metadata management** — attach structured business and technical metadata (owners, data quality, classification) to any asset using tags.
- **Governance / security** — the part this handbook focuses on: using **policy tags** to control, at query time, exactly which columns of a BigQuery table a given user is allowed to see.

Why column-level security matters

In real organizations, one table is often useful to many different teams, but not every team should see every column. A classic example: a *customers* table has `customer_id`, `city`, `email`, and `pan_number`. An analytics team needs `city` for regional reporting but has no business reason to see `pan_number` (a government ID, sensitive PII in India). Without column-level security, you'd have to either duplicate the table into a “safe” view for every access pattern, or simply not share the table at all. Policy tags let you share *one table* and let BigQuery silently enforce “who sees which column” at query time — no views, no duplicated data, no custom application code.

2. Core Concepts & Glossary

Read this section slowly if you are new to the topic — every later step in this handbook uses these exact terms. Each definition is written the way you'd explain it to someone doing this for the first time.

Term	Plain-English Definition
Data Catalog / Knowledge Catalog	Google Cloud's metadata management and governance service. It catalogs assets, lets you attach tags, and is the home of taxonomies and policy tags used for BigQuery column security. Now branded “Knowledge Catalog” (part of Dataplex Universal Catalog).
Entry	A single item in the catalog representing one data asset — e.g. a BigQuery table, a Cloud Storage fileset, or a Pub/Sub topic. Entries are usually created automatically when the catalog scans your project.
Entry Group	A logical folder that groups related entries together, mainly used for custom (non-auto-discovered) entries.
Tag Template	A reusable schema (like a form) that defines what fields a business/technical tag can hold — e.g. a template called 'data_governance' with fields 'data_owner' (string) and 'has_pii' (boolean). You define it once and reuse it on many tables.
Tag	An actual filled-in instance of a tag template, attached to a specific entry (table) or even a specific column. This is descriptive metadata — it does NOT by itself restrict access.
Taxonomy	A named, hierarchical collection of policy tags that represent one classification scheme, e.g. a taxonomy called 'Data Classification' containing the policy tags Public, Internal, Confidential. A taxonomy is a regional resource — it lives in one location (e.g. 'us' or 'asia-south1') and can only be applied to BigQuery tables in that same location.
Policy Tag	A single label inside a taxonomy (e.g. 'PII' or 'Financial') that you attach to a BigQuery column. Unlike a regular tag, a policy tag DOES restrict access: once access control is enforced on its taxonomy, only users with the right IAM role can read data in columns carrying that tag. Policy tags can be nested (parent/child), letting you build hierarchies like High → Credit Card, Government ID.
Column-level Access Control / Column-level Security (CLS)	The BigQuery feature that checks, at query time, whether the querying user has permission to read each tagged column. If they don't, BigQuery returns a permission error for just that column — the rest of the query can still run if it doesn't touch restricted columns.
Enforce Access Control	A toggle on a taxonomy. While OFF, policy tags are just descriptive labels with no effect on queries (useful for testing which columns would be affected before you turn on real restrictions). While ON, BigQuery actively blocks unauthorized reads of tagged columns.

Term	Plain-English Definition
Fine-Grained Reader	The IAM role (roles/datacatalog.categoryFineGrainedReader) that grants a user or group permission to read the actual, unmasked data in columns tagged with a specific policy tag. You grant this role on the policy tag itself, not on the table or dataset.
Data Masking / Dynamic Data Masking	An optional layer on top of column-level security. Instead of a flat allow/deny, you can let a broader group query a restricted column but see a masked version of the value (e.g. NULL, a hash, or only the last 4 digits) rather than being denied outright.
Masked Reader	The IAM role (roles/bigquerydatapolicy.maskedReader) granted to users who should see the masked version of a column instead of an access-denied error.
PII (Personally Identifiable Information)	Any data that can identify a specific individual — name, email, phone number, government ID (PAN/Aadhaar), biometric data, etc. PII columns are the most common candidates for policy tags.
IAM (Identity and Access Management)	Google Cloud's system for controlling who (identity: user, group, service account) can do what (role: a bundle of permissions) on which resource. Column-level security is layered IAM — it works in addition to, not instead of, normal dataset/table ACLs.
ACL (Access Control List)	The classic permission list attached to a BigQuery dataset or table controlling who can read/write it. Note: a user needs BOTH dataset-level read access AND the Fine-Grained Reader role on a policy tag to read a protected column — the two checks are independent (like needing both a building pass and a room key).
Aspect / Aspect Type	The newer metadata model used by Knowledge Catalog (replacing tag templates going forward). An Aspect Type defines a metadata schema; an Aspect is a filled-in instance attached to a catalog entry, including at column level. Existing tag templates still work and are being gradually migrated.
Dataplex	Google Cloud's unified data management platform that Data Catalog was absorbed into. Dataplex adds data quality scanning, lineage, and lake/zone organization on top of what Data Catalog used to do alone.

3. How Column-Level Security Actually Works

It helps to see the full chain from setup to query-time enforcement before you touch the console. There are two phases: **setup** (done once by an admin/data steward) and **enforcement** (happens automatically on every query).

Setup phase (one-time, by a data steward / admin)

1. Create a **taxonomy** in a region (e.g. 'us' or 'asia-south1') — a named container for your classification scheme.

2. Create **policy tags** inside it (e.g. PII, Financial, Health), optionally nested (PII → Email, PII → Government ID).

3. Open your BigQuery table's schema and **attach a policy tag to each sensitive column** (one policy tag per column, maximum).

4. Turn **Enforce access control** ON for the taxonomy. Until you do this, the tags are purely descriptive.

5. Grant the **Fine-Grained Reader** IAM role on each policy tag to the users/groups who should see that data unmasked.

Query-time enforcement (automatic, every time anyone runs a query)

When a user runs a SQL query against a table with tagged columns, BigQuery evaluates each column referenced in the query (including SELECT *, WHERE clauses, JOINs, etc.) against the user's IAM permissions:

Column has a policy tag?	User has Fine-Grained Reader on that tag?	Result
No	n/a	Column is returned normally (unaffected by policy tags)
Yes	Yes	Column is returned normally (with the real value)
Yes, and a masking rule exists	No, but user has Masked Reader	Column is returned masked (e.g. NULL, hashed, partial)
Yes	No, and no masking configured	Query fails with a permission-denied error naming that column

> **NOTE:** Column-level security is enforced **in addition to** dataset/table ACLs, not instead of them. A user needs read access to the dataset *and* the Fine-Grained Reader role on the column's policy tag. Also: **only one policy tag can be attached to a single column** (though a taxonomy can have many tags across many columns).

A worked example

Taxonomy: **Data Classification** (region: us) → Policy tags: **Public**, **Internal**, **Confidential**. Table `sales.customers` has `customer_id` (no tag), `city` (tagged Internal), and `pan_number` (tagged Confidential). Group `analysts@company.com` is granted Fine-Grained Reader on **Internal** only. When an analyst runs `SELECT * FROM sales.customers`, they get `customer_id` and `city` back, but the query errors out on `pan_number` unless they select around it — no separate view, no code change, no

data duplication required.

4. IAM Roles You Need to Know

Role (display name)	Role ID	What it lets you do
Data Catalog Policy Tag Admin	roles/datacatalog.categoryAdmin	Create, edit, and delete taxonomies and policy tags. Needed by whoever sets up the classification scheme.
Data Catalog Fine-Grained Reader	roles/datacatalog.categoryFineGrainedReader	Grants the ability to read actual (unmasked) data in columns tagged with a specific policy tag. Granted per-policy-tag, not per-table.
BigQuery Masked Reader	roles/bigquerydatapolicy.maskedReader	Lets a user query a masked column and see the masked value instead of getting denied.
Data Catalog Admin	roles/datacatalog.admin	Full control over Data Catalog resources, including taxonomies, tags, and templates.
Data Catalog Viewer	roles/datacatalog.viewer	Read-only visibility into catalog entries and tags (needed alongside BigQuery Admin/Data Owner to enable enforcement on a taxonomy).
BigQuery Data Owner / Admin	roles/bigquery.dataOwner or roles/bigquery.admin	Needed (together with a Data Catalog role) to toggle 'Enforce access control' on a taxonomy.

Rule of thumb for the demo: the **project owner / trainer** needs Policy Tag Admin + BigQuery Admin to set everything up. Each **test principal** (a second Google account, a group, or a service account) only needs Fine-Grained Reader on the specific policy tag(s) they should be able to read — nothing more.

5. Hands-On Demo — Step by Step

This demo builds a small 'employees' table with two sensitive columns (`email` and `salary`), classifies them with policy tags, enforces access control, and proves that an unauthorized principal is blocked while an authorized one can still read the data. You can do every step in the Cloud Console (click-through) or with the `gcloud` / `bq` CLI — both are shown. A ready-to-run Python/Colab notebook version of this same demo is described in Section 9.

> **NOTE:** Time required: ~30–45 minutes for a first attempt, including a short wait (up to ~30 minutes in rare cases, usually much faster) for IAM/policy-tag changes to fully propagate before testing denial.

Prerequisites

- A Google Cloud project with billing enabled (your training project, e.g. `himanshugcpproject`).
- The Cloud Shell or local machine with `gcloud` and `bq` CLI installed and authenticated (`gcloud auth login`).
- Your own account should have **Project Owner** or at least **BigQuery Admin** + **Data Catalog Policy Tag Admin** + **Data Catalog Admin** roles.
- (Optional but recommended) A second Google account or a Google Group you control, to demonstrate the 'denied' vs 'allowed' behaviour realistically.

Set shell variables (used throughout)

```
export PROJECT_ID="himanshugcpproject"
export LOCATION="us"                # taxonomy region; must match your BQ dataset re
gion
export DATASET="hr_demo"
export TABLE="employees"
export TAXONOMY_NAME="HR Data Classification"

gcloud config set project $PROJECT_ID
```

STEP 1 — Enable the required APIs

```
gcloud services enable \
  datacatalog.googleapis.com \
  bigquery.googleapis.com \
  bigquerydatapolicy.googleapis.com
```

Console alternative: *APIs & Services → Library → search for 'Google Cloud Data Catalog API' and 'BigQuery Data Policy API' → Enable.*

STEP 2 — Create a dataset and a table with sample PII

```
bq mk --location=$LOCATION --dataset $PROJECT_ID:$DATASET

bq query --use_legacy_sql=false \
"CREATE TABLE \`${PROJECT_ID}.${DATASET}.${TABLE}\` (
  employee_id INT64,
  full_name   STRING,
  department  STRING,
  email       STRING,
  salary      NUMERIC
)"

bq query --use_legacy_sql=false \
"INSERT INTO \`${PROJECT_ID}.${DATASET}.${TABLE}\` VALUES
  (1, 'Riya Sharma',  'Engineering', '[email protected]', 950000),
  (2, 'Arjun Verma',  'Sales',       '[email protected]', 720000),
  (3, 'Meera Nair',   'Finance',     '[email protected]',  880000)"
```

We will restrict `email` and `salary` — a good pairing to show both a PII tag and a Financial tag.

STEP 3 — Create a taxonomy and policy tags

Using gcloud:

```
gcloud data-catalog taxonomies create \
  --location=$LOCATION \
  --display-name="HR Data Classification" \
  --description="Classification for sensitive HR columns" \
  --activated-policy-types=FINE_GRAINED_ACCESS_CONTROL

# Note the returned taxonomy ID (last segment of the 'name' field), then:
export TAXONOMY_ID=""

gcloud data-catalog taxonomies policy-tags create \
  --taxonomy=$TAXONOMY_ID --location=$LOCATION --display-name="PII"

gcloud data-catalog taxonomies policy-tags create \
  --taxonomy=$TAXONOMY_ID --location=$LOCATION --display-name="Financial"
```

Using the Console: Search 'Policy tag taxonomies' (or, on newer projects, find it under 'Knowledge Catalog → Policy tags') → **Create taxonomy** → name it 'HR Data Classification', set Location to match your dataset → under Policy Tags, add 'PII' and 'Financial' as two top-level tags → Create.

STEP 4 — Assign policy tags to the sensitive columns

Using the Console (simplest for a first demo): BigQuery → open the `employees` table → Schema tab → **Edit schema** → select the `email` row → **Add policy tag** → choose 'PII' → repeat for `salary` with 'Financial' → Save.

Using the API/CLI: the bq CLI does not set policy tags directly on an existing column; instead you push an updated schema JSON that includes the policyTags block:

```
[
  {"name": "employee_id", "type": "INTEGER"},
  {"name": "full_name",   "type": "STRING"},
  {"name": "department",  "type": "STRING"},
  {"name": "email", "type": "STRING",
   "policyTags": {"names": ["projects/PROJECT_ID/locations/us/taxonomies/TAXONOMY_ID/
policyTags/PII_TAG_ID"]}},
  {"name": "salary", "type": "NUMERIC",
   "policyTags": {"names": ["projects/PROJECT_ID/locations/us/taxonomies/TAXONOMY_ID/
policyTags/FIN_TAG_ID"]}}
]

# save as schema.json, then:
bq update ${PROJECT_ID}:${DATASET}.${TABLE} schema.json
```

STEP 5 — Enforce access control on the taxonomy

Until this is switched on, the tags are only labels — nobody is actually blocked yet.

Console: open the taxonomy → toggle **Enforce access control** to ON. **gcloud** (REST, since there's no single flag):

```
curl -X PATCH \
  -H "Authorization: Bearer $(gcloud auth print-access-token)" \
  -H 'Content-Type: application/json' \
  -d '{"activatedPolicyTypes": ["FINE_GRAINED_ACCESS_CONTROL"]}' \
  "https://datacatalog.googleapis.com/v1/projects/${PROJECT_ID}/locations/${LOCATION}
/taxonomies/${TAXONOMY_ID}?updateMask=activatedPolicyTypes"
```

STEP 6 — Grant Fine-Grained Reader to a test principal

Grant access to only the **PII** tag (not Financial) for a second test account, so you can clearly demonstrate partial access.

```
export PII_TAG="projects/${PROJECT_ID}/locations/${LOCATION}/taxonomies/${TAXONOMY_ID
}/policyTags/PII_TAG_ID"

gcloud data-catalog taxonomies policy-tags add-iam-policy-binding \
  "$PII_TAG" \
  --member="user:[email protected]" \
  --role="roles/datacatalog.categoryFineGrainedReader"
```

Console: open the taxonomy → click the 'PII' policy tag → Info panel on the right → **Add principal** → enter the test account → role 'Data Catalog Fine-Grained Reader' → Save.

STEP 7 — Test it: query as the restricted user

Also make sure the test account has at least **BigQuery Data Viewer** on the dataset and **BigQuery Job User** on the project (column security is an extra layer on top of normal access, not a replacement for it). Then, signed in as the test account:

```
-- This works: email is PII, and the test user has Fine-Grained Reader on PII
SELECT employee_id, full_name, email FROM `PROJECT_ID.hr_demo.employees`;

-- This fails: salary is Financial, and the test user has NO access to that tag
SELECT employee_id, salary FROM `PROJECT_ID.hr_demo.employees`;
-- Expected error: Access Denied: BigQuery BigQuery: User does not have permission to
-- access policy tag ... on column salary.
```

IMPORTANT: As the project owner/admin, running the same query on `salary` will succeed if you also granted yourself Fine-Grained Reader on the Financial tag (or you have a role like Data Catalog Admin combined with FGR — note owner/admin roles alone do NOT bypass this check; you always need explicit Fine-Grained Reader on the specific tag).

6. Bonus: Dynamic Data Masking

Instead of a hard denial, you can let a broader audience query a restricted column and receive a masked value. This is useful for support/analytics teams who need to know 'a salary value exists' or 'this looks like a valid email' without seeing the real data.

Console steps

- Open the taxonomy → click the policy tag (e.g. 'Financial') → **Manage Data Policies**.
- Click **Add data policy** → choose a masking rule, e.g. `SHA-256 hash`, `Default masking value` (returns NULL/0/empty), `Email mask` (keeps domain, masks local part), or a custom rule for numeric ranges.
- Grant the `BigQuery Masked Reader` role on the policy tag to the group that should see masked values instead of an error.

```
-- With Masked Reader on the Financial tag, this now succeeds but returns a masked va
lue:
SELECT employee_id, salary FROM `PROJECT_ID.hr_demo.employees`;
-- salary column returns NULL (or a hash) instead of an access-denied error
```

NOTE: Data masking is not compatible with legacy SQL and has some limitations with materialized views and partitioned columns — test in a non-production dataset first.

7. Troubleshooting Common Errors

Symptom	Likely Cause / Fix
“User does not have permission to access policy tag...” for YOUR OWN account	You (or the admin) forgot to grant yourself Fine-Grained Reader on that specific policy tag. Project Owner does not automatically bypass column security.
Changes don't seem to take effect immediately	IAM and policy-tag changes can take a few minutes (rarely up to ~30 min) to propagate. Wait and retry before assuming something is misconfigured.
“Cannot apply policy tag: taxonomy and table are in different locations”	A taxonomy is regional. It can only be applied to a BigQuery dataset/table in the exact same location (e.g. both 'us', or both 'asia-south1'). Recreate the taxonomy in the matching region, or replicate it (see 'Managing policy tags across locations' in Google's docs).
“A column can only have one policy tag”	Each column supports exactly one policy tag. Model finer distinctions using a hierarchy (parent tag → child tags) rather than trying to attach two tags to one column.
Enforce access control toggle is greyed out / fails	You need BOTH a BigQuery role (Admin or Data Owner) AND a Data Catalog role (Admin or Viewer) to toggle enforcement. Also: you must delete any data masking policies on the taxonomy before you can turn enforcement OFF again.
Test user can see the table exists but every query fails, even on untagged columns	Check they also have basic BigQuery Data Viewer on the dataset and Job User/Data Viewer at the project level — policy tags are an additional layer, not a substitute for normal access.

8. Best Practices Checklist

- Start with a small number of top-level classes (e.g. Public / Internal / Confidential, or PII / Financial / Health) — most organizations never need more than 3–5.
- Use hierarchy: grant broad access at a parent tag, and lock down specific sensitive children individually (e.g. everyone with 'Low' access, but only Finance gets 'Low → Bank Account').
- Tag at the lowest necessary level — tag the `email` column, not the whole table, so the rest of the table stays freely queryable.
- Manage taxonomies, policy tags, and IAM bindings as code (Terraform) in production — manual console changes to access control are a governance and audit risk.
- Use monitor-only mode (tags created, access control NOT yet enforced) to see who would be affected before flipping enforcement on in a live environment.
- Combine with Sensitive Data Protection (Cloud DLP) to auto-discover which columns actually contain PII/PCI data across a whole project instead of tagging manually by guesswork.
- Remember row-level security (RLS) is a separate, complementary feature for restricting which ROWS a user sees — use RLS + column-level security together for full fine-grained control.

9. Companion Notebook & Cleanup

A ready-to-run Jupyter/Colab notebook (`data_catalog_policy_tags_demo.ipynb`) accompanies this handbook. It automates Steps 1–7 above using the Python client libraries (`google-cloud-datacatalog` and `google-cloud-bigquery`) so students can run each cell and see the taxonomy ID, policy tag IDs, and query results printed directly — useful for a live classroom walkthrough where re-typing long resource paths by hand is error-prone. It mirrors the same taxonomy/table names used in this handbook so the console and notebook views match up.

Cleanup (avoid leaving test resources behind)

```
# Remove the IAM binding on the policy tag (optional, deleting the taxonomy also remo
ves it)
gcloud data-catalog taxonomies policy-tags remove-iam-policy-binding "$PII_TAG" \
  --member="user:[email protected]" \
  --role="roles/datacatalog.categoryFineGrainedReader"

# Delete the taxonomy (this deletes all policy tags inside it and detaches them from
columns)
gcloud data-catalog taxonomies delete $TAXONOMY_ID --location=$LOCATION

# Delete the demo dataset
bq rm -r -f -d ${PROJECT_ID}:${DATASET}
```

10. Quick Reference & Further Reading

One-line summary of the whole workflow

```
Taxonomy (container) → Policy Tags (labels inside it) → attach tag to BQ column →
enforce access control on taxonomy → grant Fine-Grained Reader on the tag to allowed
 users →
BigQuery checks this automatically on every query, column by column.
```

Official documentation to bookmark

- Introduction to column-level access control — docs.cloud.google.com/bigquery/docs/column-level-security-intro
- Restrict access with column-level access control (step-by-step) — docs.cloud.google.com/bigquery/docs/column-level-security
- Best practices for using policy tags — docs.cloud.google.com/bigquery/docs/best-practices-policy-tags
- Transition from Data Catalog to Knowledge Catalog — docs.cloud.google.com/dataplex/docs/transition-to-dataplex-catalog
- Data Catalog / Dataplex Universal Catalog release notes — docs.cloud.google.com/dataplex/docs/release-notes

---

End of handbook. Pair this with the companion notebook for the live demo, and the Troubleshooting table in Section 7 during Q&A.;